Assignment 2: Model-free Reinforcement Learning

Peter Huijsen (s4516303) & Ahmed Al Kurwi (s4476581)
Group Number: 43

1 Introduction

In reinforcement learning, for many real-world environments, the dynamics of the environment are not entirely known: there are no clear transition or reward functions. As such, techniques such as Markov Decision Processes (MDPs), which utilize Generalized Policy Iteration (GPI) to compute an optimal policy for every state, are not feasible.

Instead, this paper explores a range of model-free techniques, which do not depend on knowing the dynamics of the environment. These agents can approximate the environment's dynamics through experience. An example of this is Monte Carlo (MC) agents, which explore the environment to approximate the transition and reward functions, based on episodic updates.

More advanced agents, namely Temporal Difference (TD) agents, approximate these dynamics more effectively and efficiently. This paper will examine a limited range of these agents: the Q-learning agent, which learns its state-action values by looking at the maximal next action; the SARSA agent, which learns by looking at the next action it will take; the expected SARSA agent, which learns by looking at the expected value of the next action based on its current policy; and the n-step SARSA agent, which learns by propagating rewards more actions back.

2 Methods

First, each of the agents examined in this paper will attempt to navigate a "shortcut" environment, as shown in the greedy paths in Figures 1a, 2a, 3a, and 4a. Here, the agent starts in one of two positions and must reach the goal, the green tile. Each step penalizes the agent by -1 reward, while the cliff edges, the red tiles, will penalize the agent by -100 reward. This paper will focus in the cumulative reward of the agent for each episode, that is, the combined reward for each step the agent takes until it reaches the goal.

In this environment, the agent is not aware of the environment's rules, i.e., the transition and reward functions, beforehand. As such, these agents apply variations of the model-free Temporal Difference (TD) learning algorithm. In short, a TD algorithm allows an agent to learn by trial and error, by discovering the value of different actions $Q(s, a)$ through experiencing them. Throughout this learning process, the agent has to balance exploration, i.e., trying new actions, and exploitation, i.e., choosing the best current action.

2.1 Action selection

To learn effectively, an agent must balance exploitation and exploration. To manage this, these agents implement an ϵ-greedy (behavior) policy: in a given state S_t, the agent selects the greedy action that maximizes the current expected reward. This can be described as the following [1]:

$$\pi_{\text{greedy}}(a) = \begin{cases} 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}|}, & \text{if } a = \text{argmax}_{b \in \mathcal{A}} Q(b) \\ \frac{\varepsilon}{|\mathcal{A}|}, & \text{otherwise} \end{cases} \tag{1}$$

Here, $\varepsilon \in [0, 1]$ described the exploration rate of the policy, where $\mathcal{A}$ is the set of all available actions to the agent

2.2 Q-Learning

The Q-learning agent applies an off-policy TD algorithm. Here, off-policy means that the policy used during training is different from the policy the agent should learn: the behavior and target policies. As such, the agent can learn a more optimal policy by first applying a more exploratory policy during training. An on-policy algorithm, however, would use the same behavior and target policy.

The Q-learning agent updates its Q-values based on the maximum expected reward of the next state, regardless of the action that the policy will choose next. Sutton and Barto [1] defines this update equation as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{2}$$

Here S_t is the current state and A_t is the current action, R_{t+1} is the immediate reward received, S_{t+1} is the next state, α is the learning rate, and γ is the discount factor.

2.3 SARSA

The SARSA agent (State-Action-Reward-State-Action) applies an on-policy TD method. It updates the current Q-value using the next action the agent would choose in the next state. The update equation for this is defined [1] as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right] \tag{3}$$

Here A_{t+1} is the next action. Since the SARSA agent bootstraps off the action it will take, i.e., the agent updates its value estimates based on its own current knowledge of future states, SARSA learns to account for the risk of its own ϵ-greedy exploration. This results in the agent maintaining a buffer from the hazard, making the SARSA agent safer during training: it produces a different final policy than the Q-Learning agent that can account more appropriately for the significant penalization of the cliff edges.

2.4 Expected SARSA

The expected SARSA agent applies a slight variation on the on-policy algorithm that the SARSA agent uses: it calculates the expected value of the next state, rather than relying on a single sampled action. Therefore, the update equation for this can be defined [1] as:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{4}$$

Here $\pi(a|S_{t+1})$ is the probability distribution of taking action a in the next state S_{t+1} under the behavior policy. This summation

weighs all possible actions by their likelihood of being selected, therefore getting the expected value for the next state.

2.5 n-step SARSA

Instead of updating based on the next step, the n-step SARSA agent delays the Q-value update of a state, and first takes another n-steps. As such, the agent accumulates a sequence of n rewards, based on which it can propagate rewards to states further back. For this, the agent takes the sum of rewards over n-steps, and the value of the state the agent ends up in after those n-steps $G_{t:t+n}$: the target return. The equation for this is defined [1] as:

$$G_{t:t+n} = \sum_{i=1}^{n} \gamma^{i-1} R_{t+i} + \gamma^n Q(S_{t+n}, A_{t+n}) \tag{5}$$

Here R_{t+i} are the rewards the agent collected on the way and $\gamma^n Q(S_{t+n}, A_{t+n})$, the bootstrapped value for the state the agent reaches after its n steps.

The episode ends when $t + n \geq T$, where $t + n$ represents the steps taken from the previous update, and T stands for the terminal time, or the end state. So, the summation cuts off at T, and the final bootstrap value $\gamma^n Q(S_{t+n}, A_{t+n})$ will converge to zero since there are no possible future states beyond the terminal time.

Once the target return is calculated, the Q-value of the state-action pair the agent was in n steps ago is updated. The update equation for this is [1]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[G_{t:t+n} - Q(S_t, A_t) \right] \tag{6}$$

Here the agent updates its previous estimate $Q(S_t, A_t)$ by taking a step towards the calculated n-step target return $G_{t:t+n}$ correcting its previous estimate based on the Temporal Difference error $[G_{t:t+n} - Q(S_t, A_t)]$ between the new target return and its original expectation.

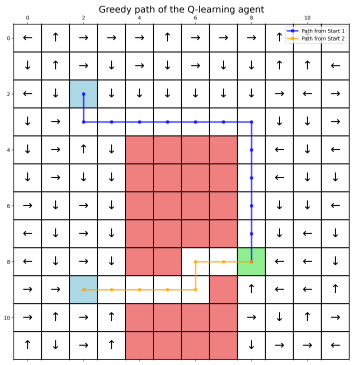
3 Results

Of the agents shown above, a number of experiments have been conducted on each to see how these agents learn, and how they perform in the introduced toy world on their own, and with respect to one another. Then, this paper will show how the Q-learning and SARSA agents perform when stochasticity is introduced into the toy environment.

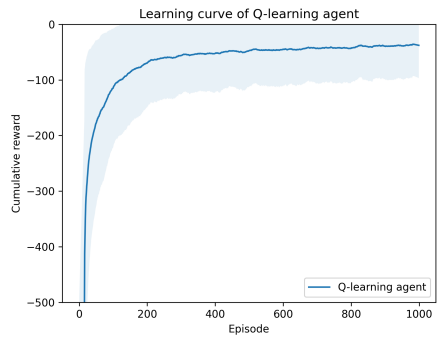
3.1 Q-learning agent

Figure 1a shows the greedy path solution for the Q-learning agent, where for both starts the agent finds the optimal policy π^*; however, the action selection of these agents, the behavior policy used during the off-policy training, follows an ε-greedy policy where the agent occasionally explores. Thus, on some episodes, the agent will take an action that results in a significant negative reward, which skews the cumulative reward for that episode.

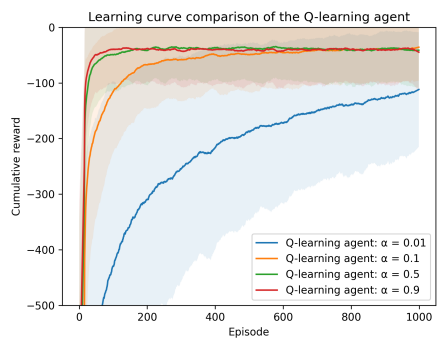
The learning curve of the Q-learning agent with $\alpha = 0.1$, and a number of learning curves at different α-values, are shown in Figure 1c. It shows that the agent learns Q_q such that $Q_q \approx Q^*$, and therefore $\pi_q \approx \pi^*$ when the agent follows a greedy policy π_q over Q_q. In early episodes, the agent often randomly ends up taking an action after which it ends up falling off the cliff edge, which penalizes the agent substantially. Note that the cumulative reward



(a) Greedy path for Q-learning with $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$ over 10,000 episodes. The path follows the cliff edge directly or goes through the cavern.



(b) Learning curve of a single experiment over 1,000 episodes, with $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$.



(c) Learning curves over 100 repetitions × 1,000 episodes, with varying α-values, $\varepsilon = 0.1$, $\gamma = 1.0$. A higher α-value decreases convergence time.

Figure 1: Q-learning agent results.

will always be negative since the agent is penalized for each action it takes, and the goal itself does not reward the agent beyond ending the episode.

Since the learned Q-values Q_q approximates Q^*, π_q will converge to π^*, which can be seen in Figure 1c; however, Figure 1c does not show an optimal cumulative reward per episode. This is explained by the agent not using a greedy policy during training, when the

cumulative reward is measured. It instead uses a ε-greedy policy. If, however, $\varepsilon \longrightarrow 0$, then the optimal cumulative reward per episode would be shown.

3.2 SARSA agent

The SARSA agent is similar to the Q-learning agent; however, it utilizes an on-policy update rule, such that there is no difference between the policy used to estimate the Q-values and the policy used for action selection. This results in a greedy path, as shown in Figure 2a, that is more reluctant to follow the cliff edge; the policy creates a buffer between itself and the cliff edge to reduce the possible unfortunate exploration actions over the cliff edge, which would penalize the agent substantially. Thus, its learned Q-values reflect its ε-greedy action selection policy.

The learning curves at different α-values in Figure 2b correspond closely with those of the Q-learning agent in Figure 1c; however, at high α-values, the SARSA agent drops in performance over the episodes, so at each update, the reward of the episode is counted relatively significantly. Therefore, volatility is increased, and, as such, one unlucky exploration encounter with the cliff on a relatively safe path, since the behavior policy is ε-greedy, will drastically skew the agent's estimate of this path.

These results are similar to those in the Cliff Walking section of Sutton and Barto [1], where the Q-learning agent follows the cliff edge, while the SARSA agent keeps a buffer between itself and the cliff edge. There is a slight difference between the learning curves, since the Q-learning agent first outperforms the SARSA agent in the "shortcut" environment; however, the resulting upper-bound performance of the SARSA agent is higher than that of the Q-learning agent, as in the example from Sutton and Barto.

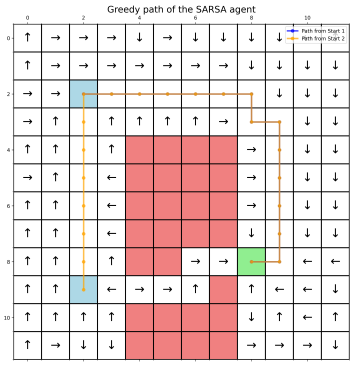
3.3 Expected SARSA agent

The expected SARSA agent combines the ideas of the Q-learning agent and the SARSA agent to produce an agent that also creates a buffer between itself and the cliff edge, as shown in Figure 3a, while maintaining its performance for high α-values, which can be seen in Figure 3b. Instead of using the next action the agent takes in the Q-value update, the agent uses the expected value, given the current policy, of the next action.

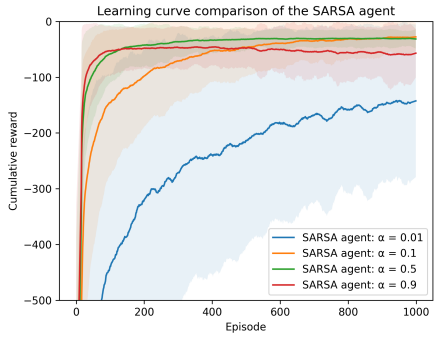
This, still on-policy, method eliminates the problem shown with the declining performance in the SARSA agent, since it weighs the cliff penalization more appropriately with the probability of taking the action that resulted in ending up in that state.

3.4 n-step SARSA agent

The previous three agents, Q-learning, SARSA, and expected SARSA, are all different versions of a 1-step TD agent. These agents all update the estimated state-action value of a state based on the single next state. Pure MC agents are ∞-step TD agents, and as such, these agents' Q-values are no longer updated based on a bootstrapped value, but on the entire episode. Now, extend this concept to n-steps back, the middle ground between these two extremes. As such, this estimate includes more information about the environment than the previous methods.



(a) Greedy path for SARSA with $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$ over 10,000 episodes. There is a clear buffer between this greedy path and the cliff edge to minimize accidental exploration actions over the cliff edge.



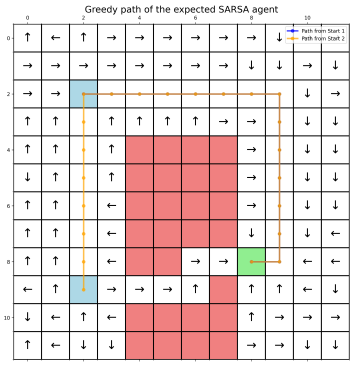
(b) Learning curves over 100 repetitions × 1,000 episodes, with varying α-values, $\varepsilon = 0.1$, $\gamma = 1.0$. Note that the single-experiment result for $\alpha = 0.1$ is included in this plot.

Figure 2: SARSA agent results.

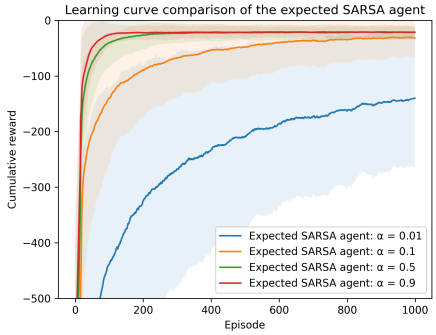
Figure 4a shows that the buffer created by the SARSA agents is now increased even further. This can be explained by the state-action values that are updated by the n-step SARSA agent with respect to the SARSA agent. The moment these agents encounter a cliff tile in their exploration, the SARSA agent updates only the state-action value of the state immediately preceding the penalized state, while the n-step SARSA agent updates all the states that were n steps back from the penalized state. Therefore, states with a higher distance to the cliff learn to avoid actions that go towards the cliff edge.

The result of this larger buffer can then be seen in Figure 4b. Updating a larger number of states preceding the penalized state lowers the overall penalization of the agent as the episodes progress. So, the cumulative reward quickly converges, where higher n-values converge faster since each update propagates more information through the state-action values; however, when $n \longrightarrow \infty$, the upper-bound of the agent's performance lowers, and, as such, for $n = 25$ the cumulative reward stabilizes at a lower value than the rest of the n-values, functioning similarly to traditional MC methods.

In addition, when the α-value increases, the variance of the agent (for $n > 1$) increases, since each update now greatly influences the



(a) Greedy path for expected SARSA with $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$ over 10,000 episodes. There is an identical buffer to the SARSA agent, shown in Figure 2b.



(b) Learning curves over 100 repetitions × 1,000 episodes, with varying α-values, $\varepsilon = 0.1$, $\gamma = 1.0$. Note that the single-experiment result for $\alpha = 0.1$ is included in this plot.

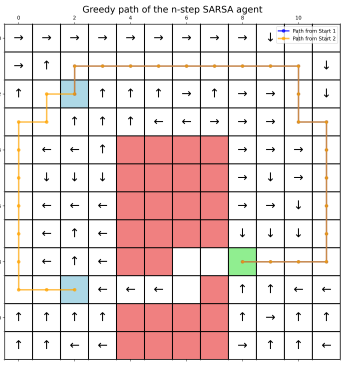
Figure 3: Expected SARSA agent results.

current Q-values, and the propagated penalization from falling off the cliff edge would stick. So, the agent's performance destabilizes, which can be seen in Figure 4c.

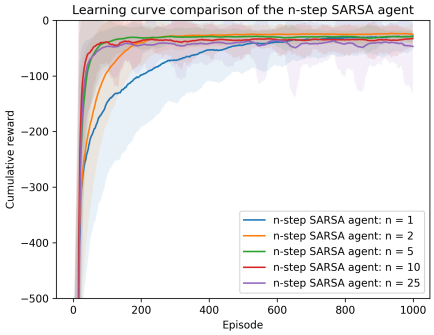
3.5 Comparison

Figure 5a shows the performance of all of the aforementioned agents. The Q-learning agent converges more rapidly than the other 1-step agents; however, its upper-bound cumulative reward is lower. Interestingly, both the SARSA and expected SARSA perform similarly at this α-value, since the expected SARSA agent performs best at higher α-values.

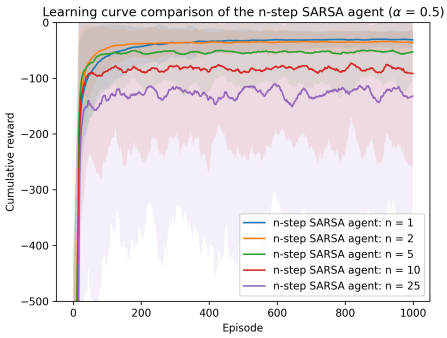
The n-step SARSA agent outperforms the other agents with a lower variance and an equal, or slightly lower, bias. This can be explained by the additional information the agent can incorporate in each update: for $n > 1$, multiple states are affected by any significant reward, which can drastically alter the state-action value estimates of states that are less directly consequential to the reward. If the learning rate were to increase, the variance of the n-step agent would prohibitively increase, and an agent with a much lower variance, such as the expected SARSA agent, could perform significantly better.



(a) Greedy path for n-step SARSA with $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$, and $n = 5$ over 10,000 episodes. The buffer from SARSA and expected SARSA, from Figures 2a & 3a, becomes larger.



(b) Learning curves over 100 repetitions × 1,000 episodes, with varying n-values, $\varepsilon = 0.1$, $\gamma = 1.0$, and $\alpha = 0.1$. At a high n-value, the agent becomes unstable, and its variance increases drastically.

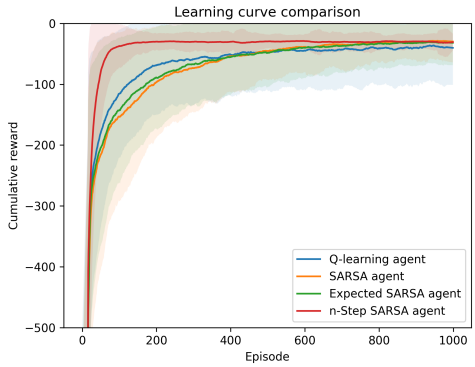


(c) Learning curves over 100 repetitions × 1,000 episodes, with varying α-values, $\varepsilon = 0.1$, $\gamma = 1.0$, and $\alpha = 0.5$ instead of $\alpha = 0.1$. The agent becomes much more unstable for high n-values.

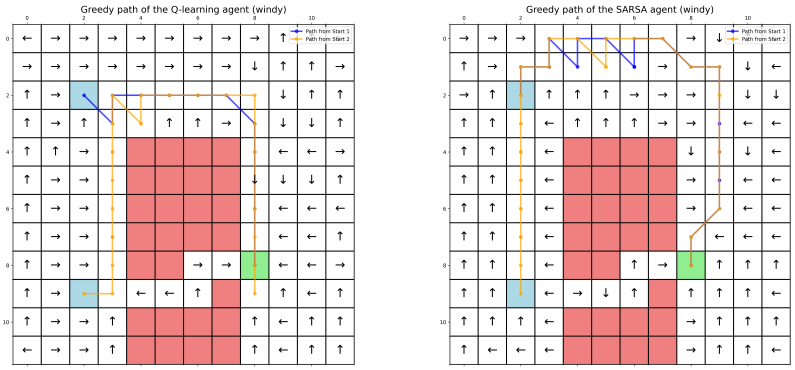
Figure 4: n-step SARSA agent results.

3.6 Windy environment

Previously, a simple environment was used without any stochasticity. That is, every action was deterministic. In Figure 5b, a variation on this environment is used where the agent is pushed down one tile with a $p = 0.5$ probability every action. This stochasticity drastically alters the strategies used by agents, since now they might



(a) Learning curves for all agents over 100 repetitions $\times$ 1,000 episodes, $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$, $n = 5$. For the n-step agent, $n = 5$ was chosen for its minimal variance with low bias, since $n = 2$ has a higher variance.



(b) Greedy paths for Q-learning (left) and SARSA (right) in the windy environment ($p = 0.5$) over 10,000 episodes, $\alpha = 0.1$, $\varepsilon = 0.1$, $\gamma = 1.0$. The introduced stochasticity and potential to be pushed over the cliff edge, incentivize the agents to keep a larger buffer to the cliff edge, to avoid an exploration step downwards, combined with being pushed downwards.

Figure 5: Comparison of all agents (left) and greedy paths in the windy environment (right).

fall off the cliff edge from an action which would otherwise result in the agent ending up in a regular tile.

The Q-learning agent no longer follows the cavern from the second start, as it did in Figure 1a, since it now risks being pushed down over the cliff edge. It now navigates around the cliff, similar to the SARSA agent, as shown in Figure 2a. In addition, a buffer is kept from the top cliff edge, since there the agent would also risk being pushed down over the cliff edge. Similarly, the SARSA agent now increases its buffer from the top cliff edge, since in some exploration steps it would randomly move down, and then also get pushed downwards. The stochasticity introduces some variance in the results, and therefore, the paths from both starts do not entirely align for either agent: sometimes the agent only moves to the right, or it is also pushed down, and therefore moves diagonally downwards.

If the wind were to come from, for instance, one of the sides, it would be expected for the agents to increase the buffer from the opposite side of the cliff edge, as they did for the wind pushing downwards. In that manner, the agents minimize the loss in cumulative reward by getting pushed over the cliff edge by the wind.

4 Discussion & Conclusion

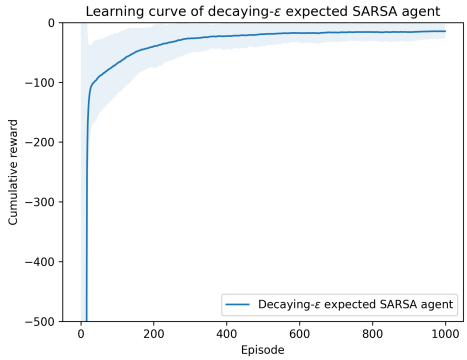
To conclude, this paper has shown the behavior and performance of four TD agents, Q-learning, SARSA, Expected SARSA, and n-step SARSA, in both a deterministic and a stochastic shortcut environment.

The one-step agents showcase the trade-off that is made between finding an optimal greedy path and maximizing cumulative reward by minimizing the possible cliff edges that an exploration action could step over into the cliff. The Q-values of the Q-learning agent rapidly converge such that the greedy path from the target policy is optimal, while the behavior policy causes the agent to occasionally step over a cliff edge, and therefore reduce cumulative reward. The SARSA agent accounts for its own random exploratory actions, learning a safer path that maintains a safe buffer from the cliff; however, this agent suffers from high variance at higher learning rates. The expected SARSA agent solves this by taking the expectation over all the possible actions in the next state. This, however, is computationally significantly more expensive than the other agents.

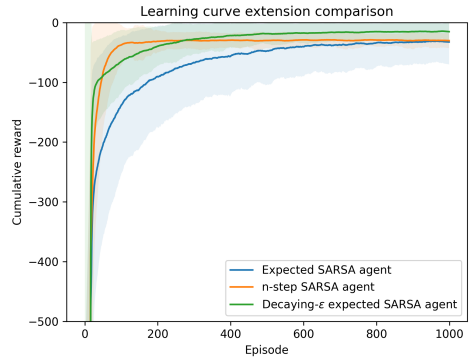
The n-step agent shows a more efficient method of updating the Q-values of these TD agents: include more information in each update by propagating rewards n steps back. As such, the agent can learn more rapidly from penalization, such as at the cliff edge, and therefore is even more cautious; its buffer to the cliff edge increases another tile with respect to the SARSA agents. As $n \longrightarrow \infty$, rewards are propagated back even further, until the update is done per episode, identically to an MC agent. Its variance increases, and its performance becomes unstable.

There are several limitations of note in this paper. First, the reliance on cumulative reward as the sole measure of performance in these experiments. Cumulative reward combines the cost of an agent's exploration with the actual quality of its learned policy. Next, this paper does not discount future rewards in any experiment; $\gamma = 1.0$, everywhere. This is appropriate for the used environment; however, this cannot simply be extrapolated to other environments in which short- vs. long-term reward prioritization is important.

Future work could better assess these algorithms to separate training performance from evaluation metrics. In addition, using a different behavior policy instead of fixed ε-greedy could significantly improve performance, such as a decaying ε-greedy policy: progressively lowering the ε-value toward zero would allow agents to transition from exploring the environment to exploiting their learned policies.



(a) The learning curve of the decaying-ε expected SARSA agent in the default environment of over 100 repetitions $\times$ 1,000 episodes, with $\alpha = 0.1$, $\varepsilon = 1.0$, $\gamma = 1.0$, and $d_r = 0.8$.



(b) Learning curves for all agents, including the decaying-ε expected SARSA agent, over 100 repetitions $\times$ 1,000 episodes, with $\alpha = 0.1$, $\varepsilon = 1.0$, $\gamma = 1.0$ and a decay rate of 0.8.

Figure 6: The learning curves of the decaying-ε expected SARSA agent (left), and a comparison of the learning curves of all agents (right).

5 Extension

Each of the agents included in the experiments in the main paper used an ε-greedy behavior policy with a fixed ε-value. This entails that each of these agents continues to explore towards the end of the 1,000 episodes for which they were run. As such, even if these agents find an optimal policy, they do not perform well on the cumulative reward measure, since they occasionally end up falling off the cliff, which heavily penalizes them. An example of this is the Q-learning agent shown in Figure 1, which finds the optimal path for both starts but which performs poorly when compared to the other agents in Figure 5a.

Therefore, this extension implements a decaying ε-greedy behavior policy for one of the earlier implemented agents. There are several ways to define this decay: linear decay, which per episode decreases ε by a fixed amount; exponential decay, which decreases ε by a factor each episode; and an amount that decreases over time, or inverse time decay. In this extension, the exponential decay option was chosen, since this seemingly would adequately balance the exploration-exploitation trade-off through the episodes. Linear decay would not decrease the ε-value fast enough, while inverse time decay would not reach a low enough ε-value.

This agent performs maximally for a maximal initial ε-value, namely $\varepsilon = 1.0$, a moderate decay rate of $d_r = 0.8$, and a low decay floor of $d_f = 0.01$. So, each episode, the ε-value of the agent decreases by 20%, relative to the previous episode, until a floor ε-value is reached. This can be represented as the following:

$$\varepsilon \leftarrow \max(d_f, \varepsilon \cdot d_r) \tag{7}$$

A too high decay rate would minimally impact ε, and therefore the performance of the agent would be more akin to that of the expected SARSA agent; a too low decay rate would prevent the agent from exploring the environment enough to find the optimal policy. The initial ε-value of $\varepsilon = 1.0$ was chosen since this maximizes the exploration of the agent at the beginning of each experiment. The agent could not have converged to the optimal policy already, so exploiting the optimal actions from our Q-table has little use.

Figure 7 shows the greedy path performance of the decaying ε expected SARSA agent, where it clearly follows the optimal path through this default environment without any stochasticity, similarly to the Q-learning agent in Figure 1a. However, its performance in terms of cumulative reward, as shown in Figure 6a, is relatively high, with a low variance and high mean. Since the agent is predominantly exploring after many episodes, the expectation would be for the agent to continuously exploit its optimal policy, and therefore achieve a consistent, high cumulative reward.

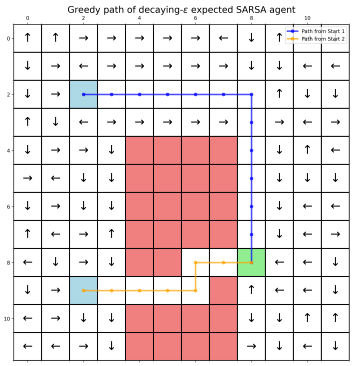


Figure 7: The greedy path of the decaying-ε expected SARSA agent, with $\alpha = 0.1$, $\varepsilon = 1.0$, $\gamma = 1.0$ and a decay rate of 0.80.

This technique would improve the online performance of not only the expected SARSA agent, as shown outperforming the other agents in Figure 6b, but also the other agents mentioned in this paper, since their policies would all converge to exploiting the optimal greedy path; no exploration eliminates the risk of falling off the cliff edge.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.